

DOCKER TASK-02

1.Create a tomcat container on 8080 and deploy sample application in tomcat.

Step 1 — Check Docker

Make sure Docker is installed and running:

```
docker --version
```

for installing: `sudo dnf install -y docker`

Check the service:

```
sudo systemctl status docker
```

If Docker is not running:

```
sudo systemctl start docker
```

```

root@ip-172-31-30-131:~
Version 2023.12.20260817:
Run the following command to upgrade to 2023.12.20260817:

dnf upgrade --releasever=2023.12.20260817

Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.12.20260817.html
=====
Installed:
container-selinux-4:2.245.0-1.amzn2023.noarch
containerd-2.2.5-1.amzn2023.0.1.x86_64
docker-25.0.16-1.amzn2023.0.3.x86_64
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
libcgroup-3.0-1.amzn2023.0.1.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
libnftnl-1.0.1-19.amzn2023.0.2.x86_64
libnftnl-1.2.2-2.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.4.x86_64
runc-1.3.5-1.amzn2023.0.2.x86_64

Complete!
[root@ip-172-31-30-131 ~]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-30-131 ~]# systemctl status docker
o docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; preset: >
   Active: inactive (dead)
   TriggeredBy: o docker.socket
   Docs: https://docs.docker.com
[root@ip-172-31-30-131 ~]# systemctl start docker
[root@ip-172-31-30-131 ~]# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; preset: >
   Active: active (running) since Mon 2026-08-24 14:32:14 UTC; 4s ago
   TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
   Process: 4199 ExecStartPre=/bin/mkdir -p /run/docker (code=exited, status=0>
   Process: 4200 ExecStartPre=/usr/libexec/docker/docker-setup-runtimes.sh (co>
   Main PID: 4201 (dockerd)
   Tasks: 9
   Memory: 31.1M
   CPU: 327ms
   CGroup: /system.slice/docker.service
           └─4201 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/cont>
Aug 24 14:32:13 ip-172-31-30-131.ec2.internal systemd[1]: Starting docker.servi>
Aug 24 14:32:13 ip-172-31-30-131.ec2.internal dockerd[4201]: time="2026-08-24T1>
Aug 24 14:32:13 ip-172-31-30-131.ec2.internal dockerd[4201]: time="2026-08-24T1>
Aug 24 14:32:14 ip-172-31-30-131.ec2.internal dockerd[4201]: time="2026-08-24T1>
Aug 24 14:32:14 ip-172-31-30-131.ec2.internal dockerd[4201]: time="2026-08-24T1>
Aug 24 14:32:14 ip-172-31-30-131.ec2.internal dockerd[4201]: time="2026-08-24T1>
Aug 24 14:32:14 ip-172-31-30-131.ec2.internal dockerd[4201]: time="2026-08-24T1>
Aug 24 14:32:14 ip-172-31-30-131.ec2.internal systemd[1]: Started docker.servi>
[root@ip-172-31-30-131 ~]#

```

Step 2 — Pull the Tomcat image

Download the Tomcat image from Docker Hub:

```
docker pull tomcat:10.1
```

Verify:

```
docker images
```

You should see something similar to:

```
REPOSITORY TAG
```

```
tomcat 10.1
```

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# docker pull tomcat:10.1  
10.1: Pulling from library/tomcat  
0926a8eb0e60: Pull complete  
89416f64c8e9: Pull complete  
fbdaa7f21d48: Pull complete  
4f4fb700ef54: Pull complete  
297337c1dc13: Pull complete  
9b67892f9c33: Pull complete  
4b9fc8fdeddd: Pull complete  
Digest: sha256:b41e73f015589a75442c98e9c12c735ad48306fd1d3ebad984cf79e290a6801a  
Status: Downloaded newer image for tomcat:10.1  
docker.io/library/tomcat:10.1  
[root@ip-172-31-30-131 ~]# docker images  
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE  
tomcat        10.1     29de696ad660   2 days ago    414MB  
[root@ip-172-31-30-131 ~]#
```

Step 3 — Create a directory for the application

Create a working directory:

```
mkdir ~/tomcat-app
```

```
cd ~/tomcat-app
```

We will create our sample application here.

Step 4 — Create the sample application

Create a simple HTML page:

```
vi index.html
```



```
cd ~/tomcat-app
```

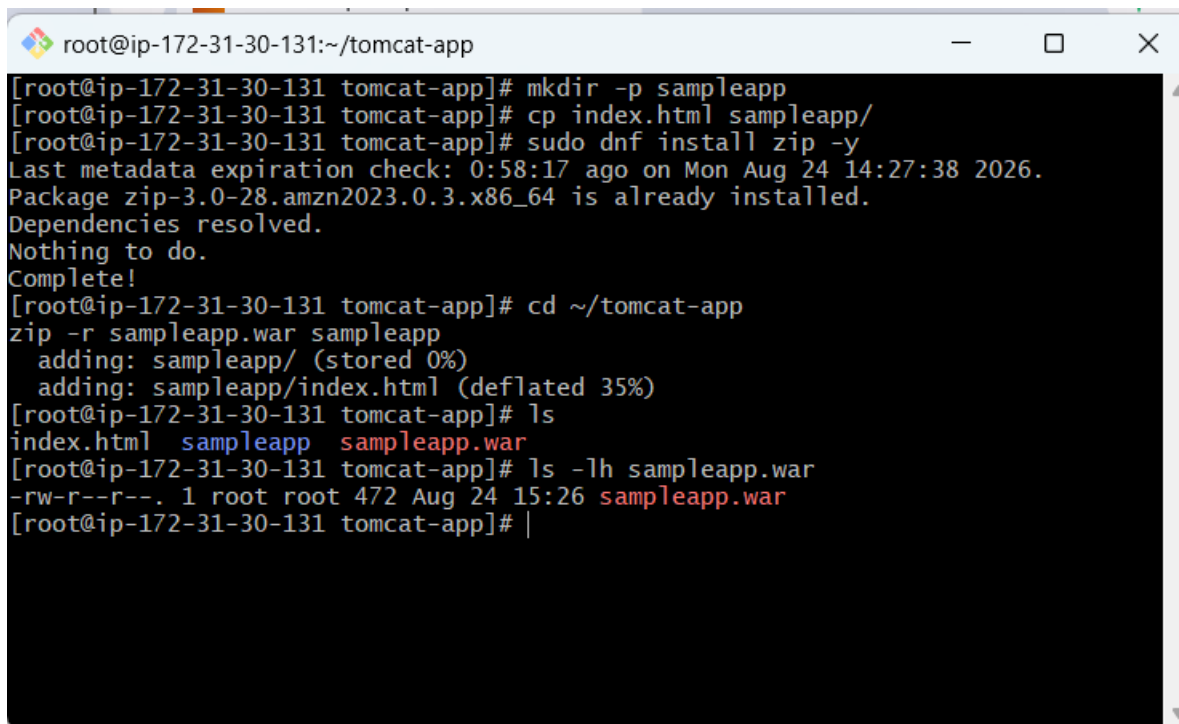
```
zip -r sampleapp.war sampleapp
```

Check:

```
ls -lh sampleapp.war
```

You should have:

sampleapp.war

A terminal window titled 'root@ip-172-31-30-131:~/tomcat-app' with standard window controls. The terminal shows the following commands and output:

```
[root@ip-172-31-30-131 tomcat-app]# mkdir -p sampleapp
[root@ip-172-31-30-131 tomcat-app]# cp index.html sampleapp/
[root@ip-172-31-30-131 tomcat-app]# sudo dnf install zip -y
Last metadata expiration check: 0:58:17 ago on Mon Aug 24 14:27:38 2026.
Package zip-3.0-28.amzn2023.0.3.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-30-131 tomcat-app]# cd ~/tomcat-app
zip -r sampleapp.war sampleapp
  adding: sampleapp/ (stored 0%)
  adding: sampleapp/index.html (deflated 35%)
[root@ip-172-31-30-131 tomcat-app]# ls
index.html  sampleapp  sampleapp.war
[root@ip-172-31-30-131 tomcat-app]# ls -lh sampleapp.war
-rw-r--r--. 1 root root 472 Aug 24 15:26 sampleapp.war
[root@ip-172-31-30-131 tomcat-app]# |
```

Step 7 — Create the Tomcat container

Now create the container:

```
docker run -d --name tomcat-server -p 8081:8080 tomcat:10.1
```

Understand -p 8080:8080

-p HOST_PORT:CONTAINER_PORT

-p 8080:8080

means:

EC2 Host

Port 8080

|

↓

Docker Container

Port 8080

|

↓

Tomcat

Step 8 — Verify that the container is running

Run:

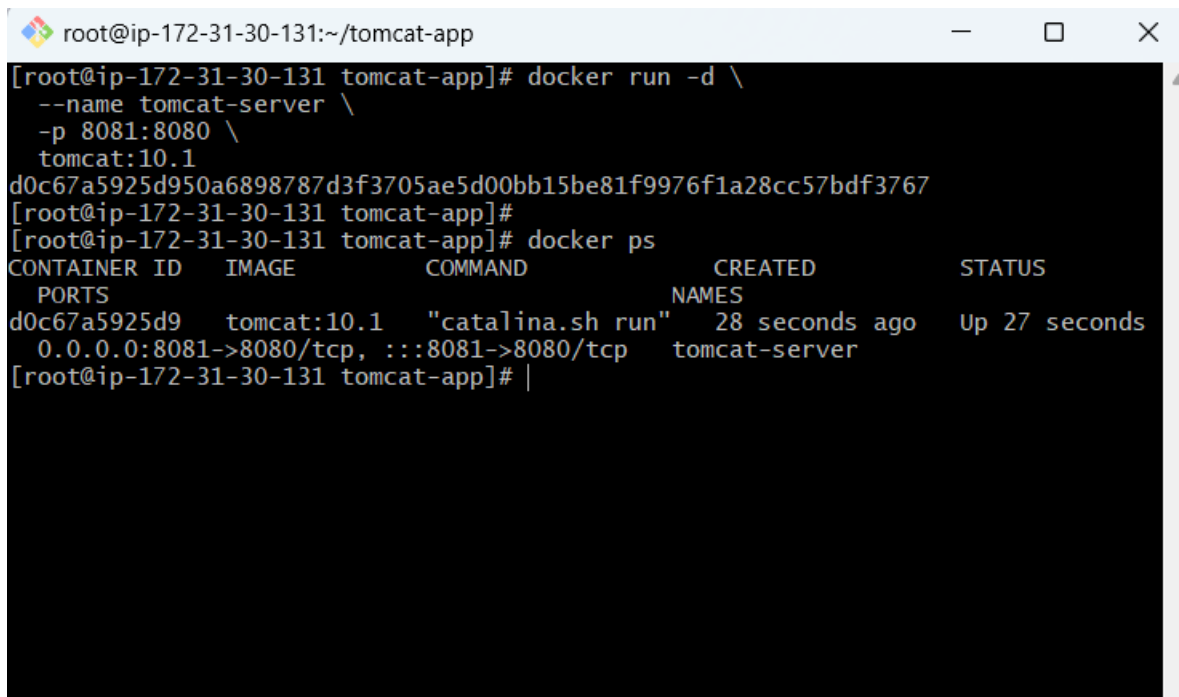
```
docker ps
```

You should see something similar to:

CONTAINER ID	IMAGE	PORTS
--------------	-------	-------

abc123	tomcat:10.1	0.0.0.0:8081->8080/tcp
--------	-------------	------------------------

This confirms that **host port 8081 is mapped to Tomcat's container port 8080**.



```
root@ip-172-31-30-131:~/tomcat-app
[root@ip-172-31-30-131 tomcat-app]# docker run -d \
--name tomcat-server \
-p 8081:8080 \
tomcat:10.1
d0c67a5925d950a6898787d3f3705ae5d00bb15be81f9976f1a28cc57bdf3767
[root@ip-172-31-30-131 tomcat-app]#
[root@ip-172-31-30-131 tomcat-app]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
d0c67a5925d9	tomcat:10.1	"catalina.sh run"	28 seconds ago	Up 27 seconds
		0.0.0.0:8081->8080/tcp, :::8081->8080/tcp	tomcat-server	

```
[root@ip-172-31-30-131 tomcat-app]# |
```

Step 9 — Check Tomcat logs

Check whether Tomcat started successfully:

```
docker logs tomcat-server
```

Look for messages indicating that Tomcat has started.

You can also follow the logs:

```
docker logs -f tomcat-server
```

Press:

Ctrl + C

to stop following the logs.

Step 10 — Copy the application into Tomcat

Tomcat's deployment directory inside the container is:

```
/usr/local/tomcat/webapps/
```

Copy the WAR file:

```
docker cp ~/tomcat-app/sampleapp.war tomcat-server:/usr/local/tomcat/webapps/
```

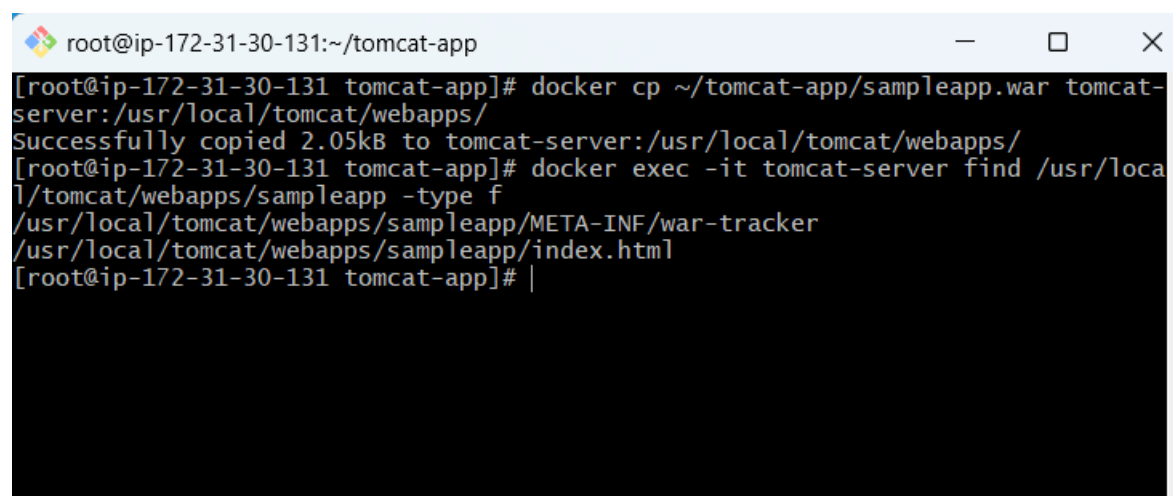
Tomcat will detect the WAR file and automatically deploy it.

Check the Tomcat logs:

```
docker logs tomcat-server
```

You can also check inside the container:

```
docker exec -it tomcat-server ls /usr/local/tomcat/webapps/
```

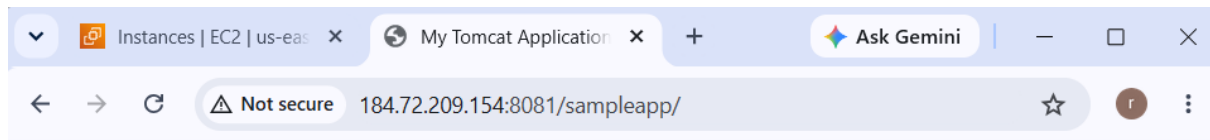
A terminal window with a title bar showing 'root@ip-172-31-30-131:~/tomcat-app'. The terminal output shows the following commands and results:

```
[root@ip-172-31-30-131 tomcat-app]# docker cp ~/tomcat-app/sampleapp.war tomcat-server:/usr/local/tomcat/webapps/
Successfully copied 2.05kB to tomcat-server:/usr/local/tomcat/webapps/
[root@ip-172-31-30-131 tomcat-app]# docker exec -it tomcat-server find /usr/local/tomcat/webapps/sampleapp -type f
/usr/local/tomcat/webapps/sampleapp/META-INF/war-tracker
/usr/local/tomcat/webapps/sampleapp/index.html
[root@ip-172-31-30-131 tomcat-app]# |
```

Step 11 — Access the application

If this is an EC2 instance, first find its public IP

<http://54.123.45.67:8080/sampleapp/>



Welcome to My Tomcat Application Rajkumari

Application deployed successfully in Docker Tomcat.

```

root@ip-172-31-30-131:~/tomcat-app
# exit
[root@ip-172-31-30-131 tomcat-app]# docker exec -it tomcat-server bash
root@d0c67a5925d9:/usr/local/tomcat# ls
bin          filtered-KEYS  native-jni-lib  RUNNING.txt    webapps.dist
BUILDING.txt lib           NOTICE         temp           work
conf         LICENSE       README.md       upstream-KEYS
CONTRIBUTING.md logs          RELEASE-NOTES   webapps
root@d0c67a5925d9:/usr/local/tomcat# cd webapps
root@d0c67a5925d9:/usr/local/tomcat/webapps# wget https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
bash: wget: command not found
root@d0c67a5925d9:/usr/local/tomcat/webapps# sudo wget https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
bash: sudo: command not found
root@d0c67a5925d9:/usr/local/tomcat/webapps# exit
exit
[root@ip-172-31-30-131 tomcat-app]# wget https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
--2026-08-24 16:12:06-- https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
Resolving tomcat.apache.org (tomcat.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to tomcat.apache.org (tomcat.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4606 (4.5K)
Saving to: 'sample.war'

sample.war          100%[=====>]  4.50K  ---KB/s   in 0s

2026-08-24 16:12:06 (45.5 MB/s) - 'sample.war' saved [4606/4606]

[root@ip-172-31-30-131 tomcat-app]# ls -lh
total 12K
-rw-r--r--. 1 root root 4.5K Mar 31 2012 sample.war
drwxr-xr-x. 2 root root  24 Aug 24 15:25 sampleapp
-rw-r--r--. 1 root root 304 Aug 24 15:59 sampleapp.war
[root@ip-172-31-30-131 tomcat-app]# docker cp sample.war tomcat-server:/usr/local/tomcat/webapps/
Successfully copied 6.14kB to tomcat-server:/usr/local/tomcat/webapps/
[root@ip-172-31-30-131 tomcat-app]# docker exec -it tomcat-server ls /usr/local/tomcat/webapps/
sample sampleapp sampleapp.war sample.war
[root@ip-172-31-30-131 tomcat-app]#

```

Instances | EC2 | us-east-1

Sample "Hello, World" Application

Ask Gemini

← → ↻

Not secure 184.72.209.154:8081/sample/

☆ ⓘ



Sample "Hello, World" Application

This is the home page for a sample application used to illustrate the source directory organization of a web application utilizing the principles outlined in the Application Developer's Guide.

To prove that they work, you can execute either of the following links:

- To a [JSP page](#).
- To a [servlet](#).



2.Create volume and deploy tomcat container on port 8081.

Step 1 — Check Docker

```
docker --version
```

Check whether Docker is running:

```
sudo systemctl status docker
```

If it is stopped:

```
sudo systemctl start docker
```

Step 2 — Pull the Tomcat image

```
docker pull tomcat:10.1
```

Verify:

```
docker images
```

You should see:

REPOSITORY	TAG
------------	-----

tomcat	10.1
--------	------

Step 3 — Create a Docker volume

Create a volume called tomcat-data:

```
docker volume create tomcat-data
```

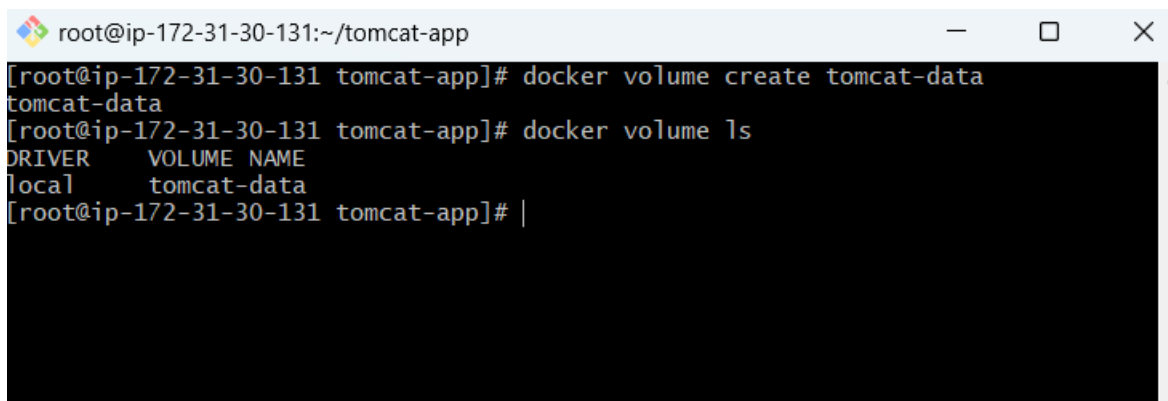
Verify:

```
docker volume ls
```

You should see:

DRIVER	VOLUME NAME
--------	-------------

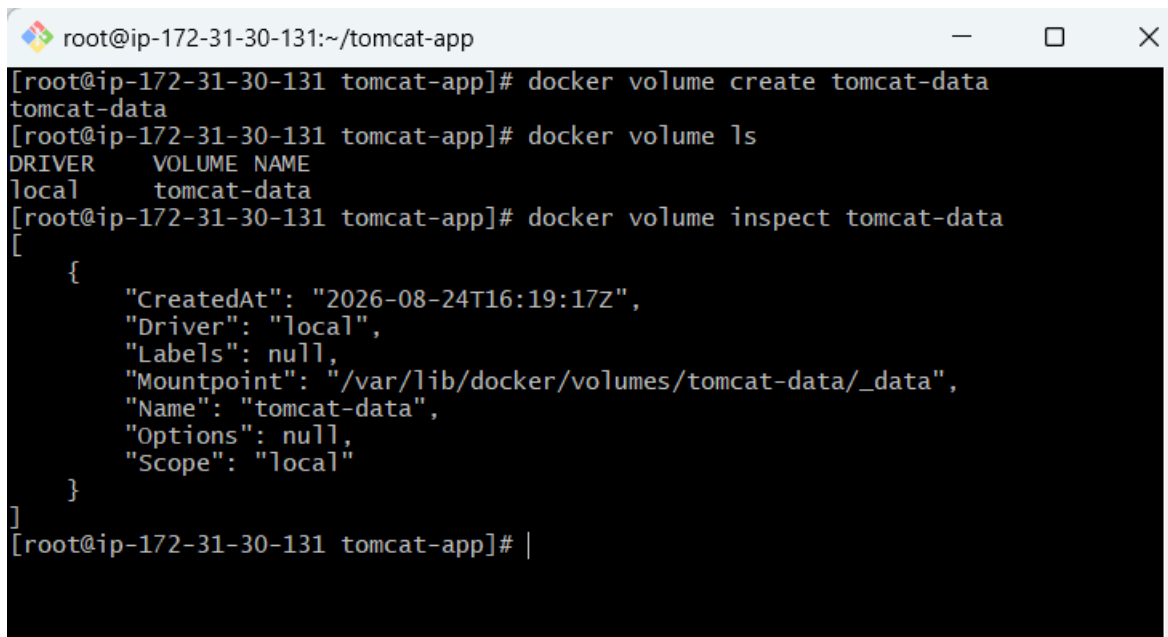
local	tomcat-data
-------	-------------

A terminal window titled 'root@ip-172-31-30-131:~/tomcat-app' showing the execution of Docker commands. The first command is 'docker volume create tomcat-data', which returns 'tomcat-data'. The second command is 'docker volume ls', which displays a table with two columns: 'DRIVER' and 'VOLUME NAME'. The table contains one entry: 'local' for 'tomcat-data'.

```
root@ip-172-31-30-131:~/tomcat-app
[root@ip-172-31-30-131 tomcat-app]# docker volume create tomcat-data
tomcat-data
[root@ip-172-31-30-131 tomcat-app]# docker volume ls
DRIVER      VOLUME NAME
local       tomcat-data
[root@ip-172-31-30-131 tomcat-app]# |
```

Step 4 — Inspect the volume

```
docker volume inspect tomcat-data
```

A terminal window titled 'root@ip-172-31-30-131:~/tomcat-app' showing the execution of Docker commands. The first command is 'docker volume create tomcat-data', which returns 'tomcat-data'. The second command is 'docker volume ls', which displays a table with two columns: 'DRIVER' and 'VOLUME NAME'. The table contains one entry: 'local' for 'tomcat-data'. The third command is 'docker volume inspect tomcat-data', which returns a JSON array containing a single object with details about the volume.

```
root@ip-172-31-30-131:~/tomcat-app
[root@ip-172-31-30-131 tomcat-app]# docker volume create tomcat-data
tomcat-data
[root@ip-172-31-30-131 tomcat-app]# docker volume ls
DRIVER      VOLUME NAME
local       tomcat-data
[root@ip-172-31-30-131 tomcat-app]# docker volume inspect tomcat-data
[
  {
    "CreatedAt": "2026-08-24T16:19:17Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/tomcat-data/_data",
    "Name": "tomcat-data",
    "Options": null,
    "Scope": "local"
  }
]
[root@ip-172-31-30-131 tomcat-app]# |
```

The **Mountpoint** is where Docker stores the volume data on the host.

Step 5 — Create the Tomcat container

Now create the container:

```
docker run -d --name tomcat-server -p 8081:8080 -v tomcat-data:/usr/local/tomcat/webapps tomcat:10.1
```

Step 6 — Verify the container

```
docker ps
```

You should see something similar to:

CONTAINER ID	IMAGE	PORTS
abc123	tomcat:10.1	0.0.0.0:8081->8080/tcp

The important part is:

```
8081->8080
```

Step 7 — Verify the volume is mounted

Run:

```
docker inspect tomcat-server
```

```
root@ip-172-31-30-131:~# docker inspect tomcat-server
[{"Id": "9a440bccefc407ceb5295160e896af0883d9ce", "Image": "tomcat:10.1", "ImageID": "sha256:9a440bccefc407ceb5295160e896af0883d9ce", "ImageName": "tomcat:10.1", "ImageVersion": "10.1", "ImageLabels": {}, "Architecture": "amd64", "Platform": "linux/amd64", "ContainerName": "tomcat-server", "Labels": {}, "Mounts": [{"Type": "volume", "Name": "tomcat-data", "Source": "/var/lib/docker/volumes/tomcat-data/_data", "Destination": "/usr/local/tomcat/webapps", "Driver": "local", "Mode": "z", "RW": true, "Propagation": ""}], "NetworkSettings": {"Networks": {"bridge": {"IP": "172.17.0.2", "Gateway": "172.17.0.1", "Subnet": "172.17.0.0/16", "MacAddress": "02:42:9d:db:1e:5f", "Name": "bridge", "NetworkID": "k8s", "EndpointID": "9a440bccefc407ceb5295160e896af0883d9ce", "InterfaceName": "eth0", "IPv6Address": ""}}, "Bridge": "bridge", "IP": "172.17.0.2", "Gateway": "172.17.0.1", "Subnet": "172.17.0.0/16", "MacAddress": "02:42:9d:db:1e:5f", "Name": "bridge", "NetworkID": "k8s", "EndpointID": "9a440bccefc407ceb5295160e896af0883d9ce", "InterfaceName": "eth0", "IPv6Address": ""}, "HostConfig": {"Binds": ["/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin", "/opt/java/openjdk:/opt/java/openjdk", "/usr/local/sbin:/usr/local/sbin:/usr/local/sbin:/usr/local/bin:/usr/local/sbin:/usr/local/bin"], "ContainerIDFile": "", "LogConfig": {"Type": "json-file", "Config": {}}, "NetworkMode": "bridge", "PortBindings": {"8080/tcp": [{"HostPort": "8081", "ContainerPort": "8080"}]}, "RestartPolicy": {"Name": "no", "Condition": "never"}, "SecurityOpt": ["seccomp=unconfined"], "StorageOpt": "overlay2", "Sysctl": {}, "Tmpfs": {}, "Uts": {"Id": "9a440bccefc407ceb5295160e896af0883d9ce", "Name": "9a440bccefc407ceb5295160e896af0883d9ce", "NetworkID": "k8s", "EndpointID": "9a440bccefc407ceb5295160e896af0883d9ce", "InterfaceName": "eth0", "IPv6Address": ""}, "VolumeMounts": [{"Type": "volume", "Name": "tomcat-data", "Source": "/var/lib/docker/volumes/tomcat-data/_data", "Destination": "/usr/local/tomcat/webapps", "Driver": "local", "Mode": "z", "RW": true, "Propagation": ""}], "WorkingDir": "/usr/local/tomcat/work"}]
```

Step 8 — Check Tomcat inside the container

Enter the container:

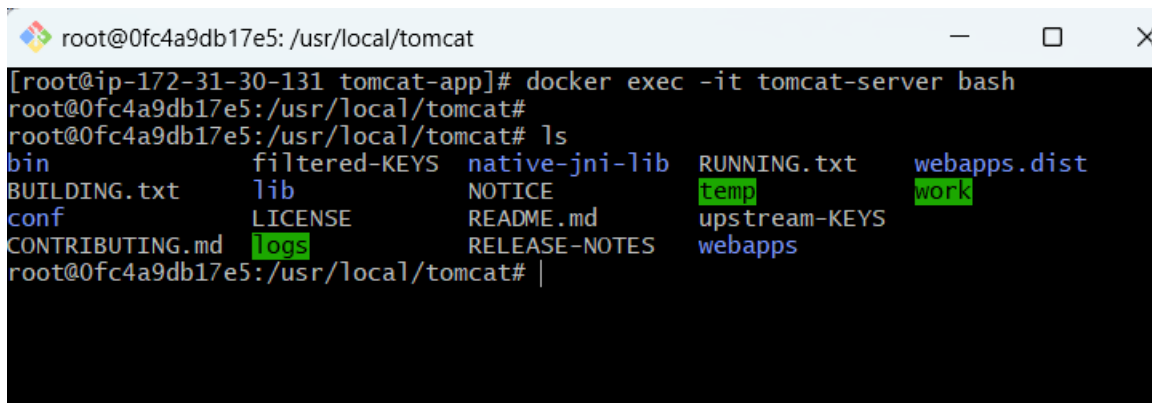
```
docker exec -it tomcat-server bash
```

Check the webapps directory:

```
ls -la /usr/local/tomcat/webapps
```

Then exit:

```
exit
```



```
root@0fc4a9db17e5: /usr/local/tomcat
[root@ip-172-31-30-131 tomcat-app]# docker exec -it tomcat-server bash
root@0fc4a9db17e5:/usr/local/tomcat#
root@0fc4a9db17e5:/usr/local/tomcat# ls
bin                filtered-KEYS      native-jni-lib    RUNNING.txt       webapps.dist
BUILDING.txt       lib               NOTICE           temp              work
conf              LICENSE           README.md         upstream-KEYS
CONTRIBUTING.md   logs             RELEASE-NOTES     webapps
root@0fc4a9db17e5:/usr/local/tomcat# |
```

Step 9 — Test Tomcat locally

From the Linux host:

```
curl http://localhost:8081
```

If Tomcat is working, you should receive the Tomcat response.

You can also open:

```
http://<EC2-PUBLIC-IP>:8081
```

in your browser.

Step 10 — Allow port 8081 in AWS Security Group

If this is an EC2 instance, add an inbound rule:

Type: Custom TCP

Port: 8081

Source: Your IP

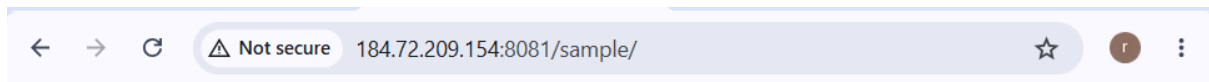
For temporary testing, you can use:

```
0.0.0.0/0
```

but restricting it to your IP is safer.

Then open:

http://<EC2-PUBLIC-IP>:8081



Sample "Hello, World" Application

This is the home page for a sample application used to illustrate the source directory organization of a web application utilizing the principles outlined in the Application Developer's Guide.

To prove that they work, you can execute either of the following links:

- To a [JSP page](#).
- To a [servlet](#).

3. Limit the nginx container to 500 MB.

Step-by-step

1. Pull the Nginx image

```
docker pull nginx
```

2. Create the Nginx container with 500 MB memory limit

```
docker run -d \  
  --name nginx-server \  
  --memory=500m \  
  nginx
```

The important option is:

```
--memory=500m
```

It means the container can use a maximum of approximately **500 MB of RAM**.

3. Verify the container

docker ps

You should see:

nginx-server

```
root@ip-172-31-30-131:~/tomcat-app
[root@ip-172-31-30-131 tomcat-app]# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
26c307b5e35a: Pull complete
746b934a8960: Pull complete
5508f6432d3e: Pull complete
5d480233f531: Pull complete
f530c3e421fc: Pull complete
128fcc7b23b0: Pull complete
7eb55399d6de: Pull complete
Digest: sha256:0d4374c710a9649200e84f8ef8dbdd4fa76c0c107839cd50f1e42a63916b0f2e
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
[root@ip-172-31-30-131 tomcat-app]# docker run -d --name nginx-server --memory=500m nginx
684cceeefb8530c7bbf6d3430c0625819752292976335348d35c129d2dcf85fa0
[root@ip-172-31-30-131 tomcat-app]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
684cceeefb853  nginx         "/docker-entrypoint...."  14 seconds ago Up 13 seconds  80/tcp
0fc4a9db17e5   tomcat:10.1   "catalina.sh run"        32 minutes ago Up 32 minutes  0.0.0.0:808
1->8080/tcp, :::8081->8080/tcp  tomcat-server
[root@ip-172-31-30-131 tomcat-app]# docker stats nginx-server
CONTAINER ID   NAME          CPU %   MEM USAGE / LIMIT   MEM %   NET I/O   BLOCK I/O
```

4. Check using docker stats

Run:

docker stats nginx-server

You might see:

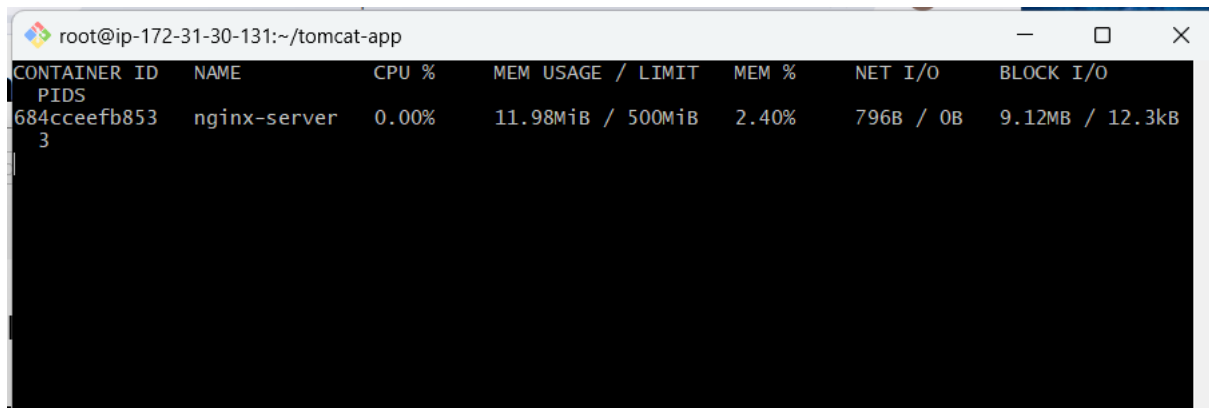
CONTAINER CPU % MEM USAGE / LIMIT

nginx-server 0.00% 1.5MiB / 500MiB

Here:

MEM USAGE → Current memory being used

LIMIT → Maximum allowed memory



A terminal window titled 'root@ip-172-31-30-131:~/tomcat-app' displays the output of the 'docker ps' command. The output shows a single container named 'nginx-server' with ID '684cceedfb853'. The container is running and has 3 PIDs. The table below summarizes the data shown in the terminal.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O
684cceedfb853	nginx-server	0.00%	11.98MiB / 500MiB	2.40%	796B / 0B	9.12MB / 12.3kB

4. Create a sample docker file using below instructions.

- 1. Base module as amazonlinux:latest**
- 2. Maintainer you name**
- 3. Install nginx**
- 4. COPY one index.html file to image**
- 5. Expose on port 80**
- 6. Command to start the nginx container**

1. Create a working directory

```
mkdir nginx-docker
```

```
cd nginx-docker
```

2. Create index.html

```
vi index.html
```

```
root@ip-172-31-30-131:~/nginx-docker
<!DOCTYPE html>
<html>
<head>
  <title>My Nginx Application</title>
</head>
<body>
  <h1>Welcome to Nginx Docker Container Rajkumari</h1>
  <p>This page is running on Amazon Linux.</p>
</body>
</html>
```

3. Create the Dockerfile

vi Dockerfile

```
root@ip-172-31-30-131:~/nginx-docker
FROM amazonlinux:latest
MAINTAINER Rajkumari
RUN dnf install -y nginx
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
-- INSERT --
```

Explanation line by line

Dockerfile instruction	Purpose
FROM amazonlinux:latest	Uses Amazon Linux as the base image
MAINTAINER maintainer="Your Name"	Specifies the maintainer
RUN dnf install -y nginx	Installs Nginx
COPY index.html ...	Copies your HTML file into Nginx's web directory

Dockerfile instruction

Purpose

EXPOSE 80

Documents that Nginx listens on port 80

CMD [...]

Starts Nginx when the container starts

Note: LABEL maintainer is preferred over the old MAINTAINER instruction because MAINTAINER is deprecated.

5. Build the Docker image

`docker build -t my-nginx-image`

Here:

`-t my-nginx:1.0`

↑

Image name : tag

Check the image:

`docker images`

```
root@ip-172-31-30-131:~/nginx-docker
[root@ip-172-31-30-131 nginx-docker]# docker build -t my-nginx-image .
[+] Building 32.5s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 268B
=> [internal] load metadata for docker.io/library/amazonlinux:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/3] FROM docker.io/library/amazonlinux:latest@sha256:181f98c48832fe926f8ca3b6ffea9c128e96e77b93d08f8e9a9bc9403ce284
=> [internal] load build context
=> => transferring context: 92B
=> [2/3] RUN dnf install -y nginx
=> [3/3] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => writing image sha256:6cf6a08a2fc99a7d0f0adbe740460d25ffdbdecae09a56bb852f5b2f67d65795
=> => naming to docker.io/library/my-nginx-image
[root@ip-172-31-30-131 nginx-docker]#
```

```
root@ip-172-31-30-131:~
[root@ip-172-31-30-131 ~]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
my-nginx-image       latest      6cf6a08a2fc9  11 minutes ago 388MB
tomcat               10.1       29de696ad660  3 days ago    414MB
nginx                latest      f075e3f94986  5 days ago    162MB
[root@ip-172-31-30-131 ~]#
```

7. Create and start the container

Run:

```
docker run -d --name my-nginx-container -p 80:80 my-nginx-image
```

Explanation:

-d → Run container in background

--name → Container name

-p 80:80 → Host port 80 → Container port 80

my-nginx-image → Image used to create container

Check the container:

```
docker ps
```

You should see:

CONTAINER ID	IMAGE	PORTS
--------------	-------	-------

```
root@ip-172-31-30-131:~# docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
my-nginx-image latest    6cf6a08a2fc9   11 minutes ago 388MB
tomcat         10.1     29de696ad660   3 days ago    414MB
nginx          latest    f075e3f94986   5 days ago    162MB
[root@ip-172-31-30-131 ~]# docker run -d --name my-nginx-container -p 80:80 my-nginx-image
cb9a128373f34d91fbf332e5b243deb8f27218c4eec087eddc49860d5e84ea37
[root@ip-172-31-30-131 ~]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
cb9a128373f3   my-nginx-image nginx -g 'daemon off;'   14 seconds ago Up 13 seconds 0.0.0.0:80->80/tcp, :::80->80/tcp   my-nginx-container
[root@ip-172-31-30-131 ~]#
```

8. Test Nginx

On your EC2 server, run:

```
curl http://localhost
```

You should get your HTML content:

```
<h1>Hello from Nginx Docker Container!</h1>
```

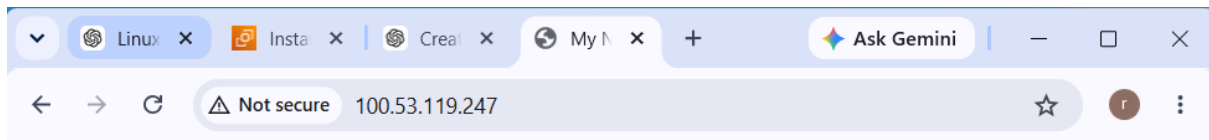
If you are using an **AWS EC2 instance**, open:

```
http://<EC2-PUBLIC-IP>
```

For example:

```
http://54.xx.xx.xx
```

Make sure the EC2 **Security Group** allows inbound TCP port 80.



Welcome to Nginx Docker Container Rajkumari

This page is running on Amazon Linux.

5. Push image to Docker hub

1. Check your image

docker images

You should see:

REPOSITORY	TAG	IMAGE ID
my-nginx-image	latest	xxxxxxxxxx

```
root@ip-172-31-30-131:~/nginx-docker
[root@ip-172-31-30-131 nginx-docker]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
my-nginx-image       latest             6cf6a08a2fc9       42 minutes ago    388MB
tomcat               10.1              29de696ad660       3 days ago        414MB
nginx                latest            f075e3f94986       5 days ago        162MB
[root@ip-172-31-30-131 nginx-docker]# |
```

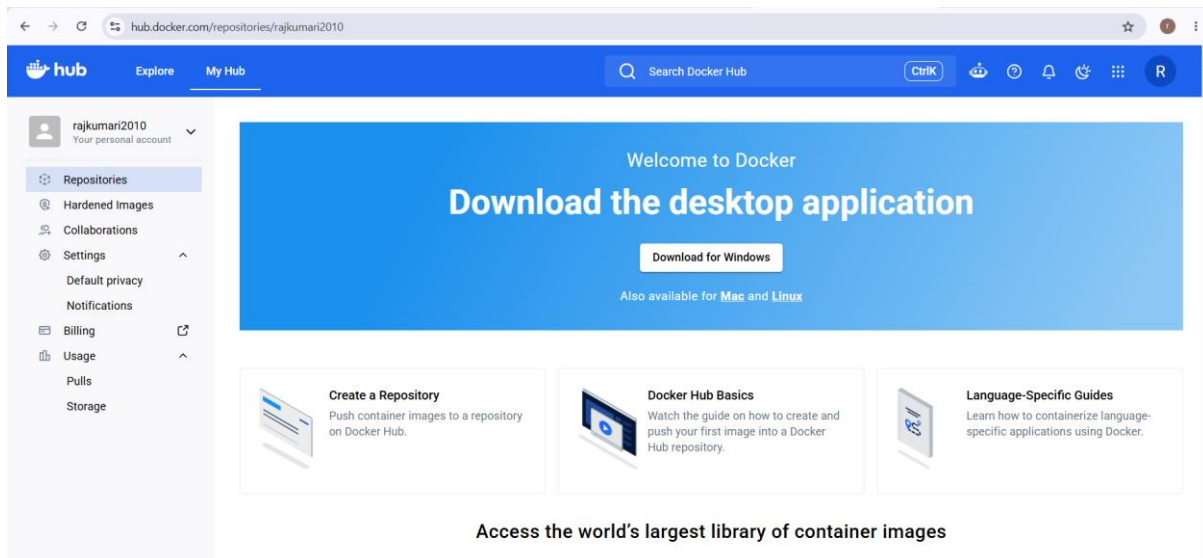
2. Login to Docker Hub

docker login

Enter your **Docker Hub username** and password/PAT when prompted.

If successful:

Login Succeeded



3. Tag your image

Docker Hub images normally use this format:

DOCKERHUB_USERNAME/REPOSITORY_NAME:TAG

For example, if your Docker Hub username is rajkumari:

`docker tag my-nginx-image:latest rajkumari/my-nginx-image:latest`

Check:

`docker images`

You should now see:

`rajkumari/my-nginx-image latest`

`my-nginx-image latest`

```
root@ip-172-31-30-131:~/nginx-docker
[root@ip-172-31-30-131 nginx-docker]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
my-nginx-image      latest     6cf6a08a2fc9  42 minutes ago 388MB
tomcat              10.1       29de696ad660  3 days ago    414MB
nginx               latest     f075e3f94986  5 days ago    162MB
[root@ip-172-31-30-131 nginx-docker]# docker tag my-nginx-image:latest rajkumari/my-nginx-image:latest
[root@ip-172-31-30-131 nginx-docker]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
my-nginx-image      latest     6cf6a08a2fc9  47 minutes ago 388MB
rajkumari/my-nginx-image latest     6cf6a08a2fc9  47 minutes ago 388MB
tomcat              10.1       29de696ad660  3 days ago    414MB
nginx               latest     f075e3f94986  5 days ago    162MB
[root@ip-172-31-30-131 nginx-docker]#
```

4. Push the image

`docker push rajkumari/my-nginx-image:latest`

You will see layers being uploaded:

The push refers to repository [docker.io/rajkumari/my-nginx-image]

latest: digest: sha256:xxxxxxx...

```
root@ip-172-31-30-131:~/nginx-docker
[root@ip-172-31-30-131 nginx-docker]# docker info | grep Username
[root@ip-172-31-30-131 nginx-docker]# echo $?
1
[root@ip-172-31-30-131 nginx-docker]# docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and is required for organizations
at https://docs.docker.com/go/access-tokens/

Username: rajkumari2010
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

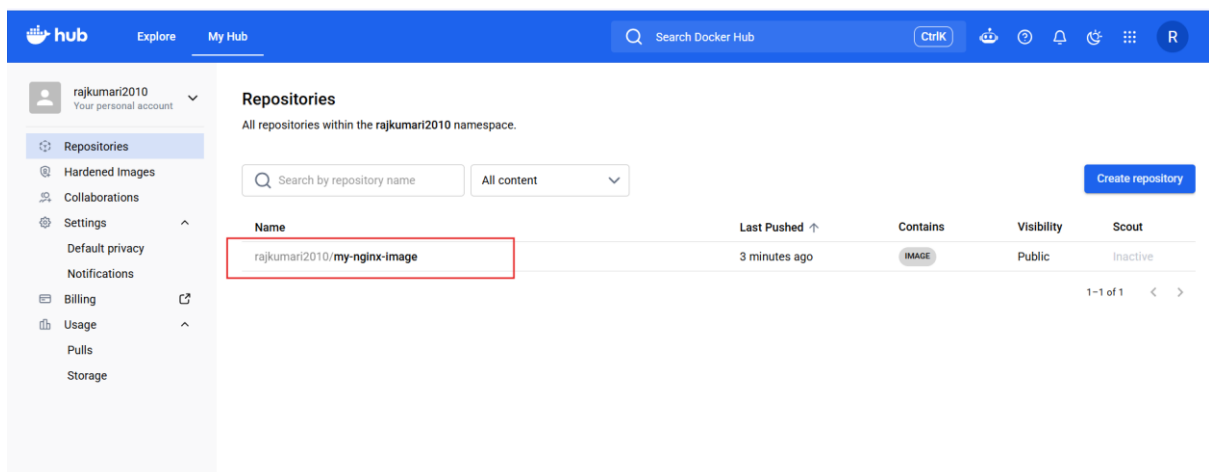
Login Succeeded
[root@ip-172-31-30-131 nginx-docker]# docker info | grep Username
Username: rajkumari2010
[root@ip-172-31-30-131 nginx-docker]# docker push rajkumari2010/my-nginx-image:latest
The push refers to repository [docker.io/rajkumari2010/my-nginx-image]
d0748c812789: Pushed
08d3bce09da0: Pushed
bba25e797da6: Mounted from library/amazonlinux
latest: digest: sha256:559c1242b4b237d6f2ee2de7313e93b7df08494da389cc893c052b22d84cc0c1 size: 949
[root@ip-172-31-30-131 nginx-docker]#
```

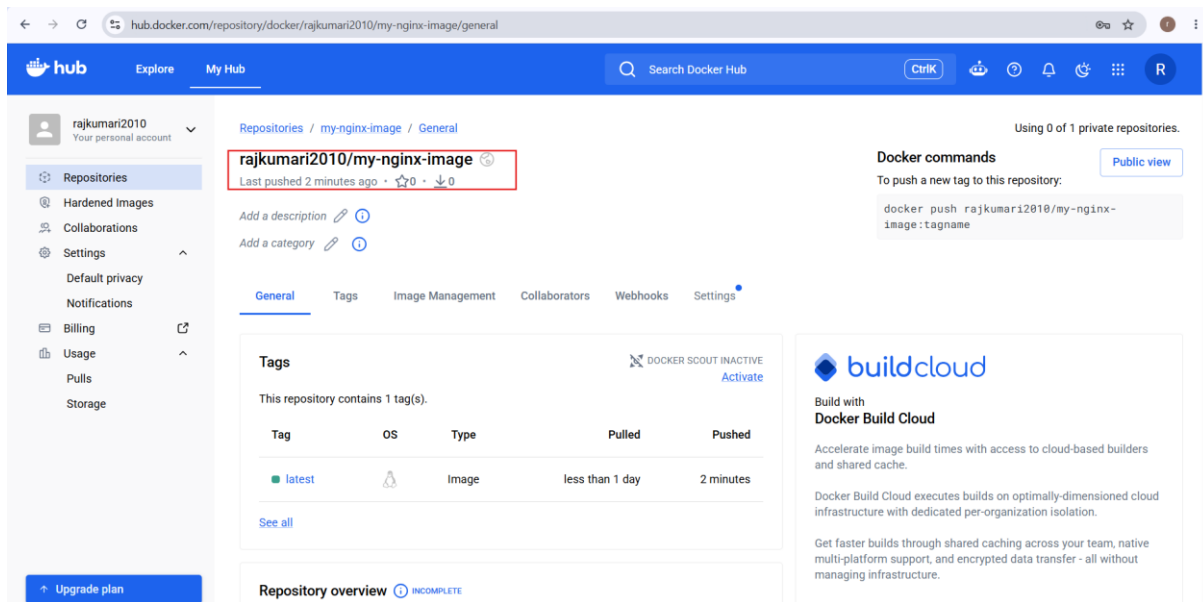
5. Verify on Docker Hub

Go to Docker Hub and open your repository.

You should find:

rajkumari/my-nginx-image





6. Test by pulling the image

You can test from another machine:

```
docker pull rajkumari/my-nginx-image:latest
```

Then run it:

```
docker run -d --name test-nginx -p 80:80 rajkumari/my-nginx-image:latest
```

6. push image to AWS ECR

Assume:

- AWS Region: us-east-1
- Image: my-nginx-image
- ECR repository: my-nginx-repo

1. Create an ECR repository

```
aws ecr create-repository \  
  --repository-name my-nginx-repo \  
  --region us-east-1
```

Check:

```
aws ecr describe-repositories \  
  --repository-names my-nginx-repo \
```

--region us-east-1

Attach an IAM role to this EC2 instance with ECR permissions.

1. AWS Console → **IAM** → **Roles** → **Create role**
2. Trusted entity: **AWS service**
3. Use case: **EC2**
4. Attach:

AmazonEC2ContainerRegistryPowerUser

5. Create the role, e.g.:

EC2-ECR-Role

6. Go to **EC2** → **Instances** → **your instance**
7. **Actions** → **Security** → **Modify IAM role**
8. Select EC2-ECR-Role
9. Save.

The screenshot shows the AWS IAM console 'Create role' page, specifically Step 2: 'Add permissions'. The page is titled 'Create role' and shows the breadcrumb 'IAM > Roles > Create role'. On the left, a progress bar indicates three steps: 'Step 1: Select trusted entity', 'Step 2: Add permissions' (which is the current step), and 'Step 3: Name, review, and create'. The main content area is titled 'Add permissions' and has two options: 'Use existing policy' (selected) and 'Create inline policy'. Below these options, there is a section for 'Permissions policies (1/1219)'. A search bar contains 'AmazonEC2ContainerRegistryPowerUser' and shows '1 match'. A table lists the policy 'AmazonEC2ContainerRegistryPowerUser' as an 'AWS managed' policy with the description 'Provides full access to Amazon EC2 Container Registry repositories, but does n...'. At the bottom, there is a link to 'Set permissions boundary - optional' and navigation buttons for 'Cancel', 'Previous', and 'Next'.

Step 1
Select trusted entity

Step 2
Add permissions

Step 3
Name, review, and create

Add permissions info

☒ Use existing policy
Attach an existing managed policy to the role.

☐ Create inline policy
Create a new policy that will be attached directly to the role.

Permissions policies (1/1219) info

Choose one or more policies to attach to your new role.

Filter by Type: All types 1 match

Policy name	Type	Description
☒ AmazonEC2ContainerRegistryPowerUser	AWS managed	Provides full access to Amazon EC2 Container Registry repositories, but does n...

► Set permissions boundary - optional

Cancel Previous Next

aws

Search

[Alt+S]

Global

Rajkumari (571833381790)

Rajkumari

IAM > Roles > Create role

Step 1
Select trusted entity

Step 2
Add permissions

Step 3
Name, review, and create

Name, review, and create

Role details

Role name

Enter a meaningful name to identify this role.

EC2-ECR-Role

Maximum 64 characters. Use alphanumeric and '+', '@', '-' characters.

Description

Add a short explanation for this role.

Allows EC2 instances to call AWS services on your behalf.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: '_', '+', '@', '-', '/', '[', ']', '#', '%', '*', ':', '!', '~', '.', '"', ''', '`', '<', '>', '?', '@', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', '[', '\', ']', '^', '_', '`', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', '{', '|', '}', '~', '', '€', '', '‚', 'ƒ', '„', '…', '†', '‡', 'ˆ', '‰', 'Š', '‹', 'Œ', '', 'Ž', '', '', '‘', '’', '“', '”', '•', '–', '—', '˜', '™', 'š', '›', 'œ', '', 'ž', 'Ÿ', ' ', '¡', '¢', '£', '¤', '¥', '¦', '§', '¨', '©', 'ª', '«', '¬', '­', '®', '¯', '°', '±', '²', '³', '´', 'µ', '¶', '·', '¸', '¹', 'º', '»', '¼', '½', '¾', '¿', 'À', 'Á', 'Â', 'Ã', 'Ä', 'Å', 'Æ', 'Ç', 'È', 'É', 'Ê', 'Ë', 'Ì', 'Í', 'Î', 'Ï', 'Ð', 'Ñ', 'Ò', 'Ó', 'Ô', 'Õ', 'Ö', '×', 'Ø', 'Ù', 'Ú', 'Û', 'Ü', 'Ý', 'Þ', 'ß', 'à', 'á', 'â', 'ã', 'ä', 'å', 'æ', 'ç', 'è', 'é', 'ê', 'ë', 'ì', 'í', 'î', 'ï', 'ð', 'ñ', 'ò', 'ó', 'ô', 'õ', 'ö', '÷', 'ø', 'ù', 'ú', 'û', 'ü', 'ý', 'þ', 'ÿ'.

Step 1: Select trusted entities

Trust policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "ecr:BatchGetImage"
```

aws

Search

[Alt+S]

Global

Rajkumari (571833381790)

Rajkumari

IAM > Roles

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles

Role EC2-ECR-Role created.

View role

Delete

Create role

Roles (1/11) Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

Role name

Trusted entities

Last activity

aws

Search

[Alt+S]

Global

Rajkumari (571833381790)

Rajkumari

EC2 > Instances > i-092add6ce1e26dacf > Modify IAM role

Modify IAM role Info

Attach an IAM role to your instance.

Instance ID

i-092add6ce1e26dacf (Tocat-Server)

IAM role

Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.

EC2-ECR-Role

Create new IAM role

Cancel

Update IAM role

Then return to your EC2 terminal and run:

```
aws sts get-caller-identity
```

You should get an account ID and a role ARN.

```
root@ip-172-31-30-131:~/nginx-docker
[root@ip-172-31-30-131 nginx-docker]# aws sts get-caller-identity
{
  "UserId": "AROAYKI7M6OPWVOJDCRB:i-092add6ce1e26dacf",
  "Account": "571833381790",
  "Arn": "arn:aws:sts::571833381790:assumed-role/EC2-ECR-Role/i-092add6ce1e26dacf"
}
[root@ip-172-31-30-131 nginx-docker]# |
```

Add ECR permissions to EC2-ECR-Role

Go to:

AWS Console → IAM → Roles → EC2-ECR-Role

Click:

Add permissions → Attach policies

For your lab, attach:

AmazonEC2ContainerRegistryFullAccess

This gives the role the ECR permissions needed for pushing images.

If you already attached this policy, check whether the policy is actually attached to EC2-ECR-Role and not to a different role.

aws | IAM > Roles > EC2-ECR-Role > Add Permissions

Attach policy to EC2-ECR-Role

► **Current permissions policies (1)**

Other permissions policies (1/1218)

Filter by Type

AmazonEC2ContainerRegistry All types 3 matches

Policy name	Type
☒ AmazonEC2ContainerRegistryFullAccess	AWS managed
☐ AmazonEC2ContainerRegistryPullOnly	AWS managed
☐ AmazonEC2ContainerRegistryReadOnly	AWS managed

Cancel Add permissions

aws | IAM > Roles > EC2-ECR-Role

Permissions policies (2)

You can attach up to 20 managed policies.

Filter by Type

Policy name	Type	Attached entities
☐ AmazonEC2ContainerRegistryFullAccess	AWS managed	1
☐ AmazonEC2ContainerRegistryPowerUser	AWS managed	1

► **Permissions boundary (not set)**

▼ **Generate policy based on CloudTrail events**

You can generate a new policy based on the access activity for this role, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

[Generate policy](#)

No requests to generate a policy in the past 7 days.

Go back to EC2

You don't need to recreate the EC2 instance or restart it.

Run:

```
aws sts get-caller-identity
```

You should still see:

arn:aws:sts::571833381790:assumed-role/EC2-ECR-Role/...

Step 3 — Create the ECR repository

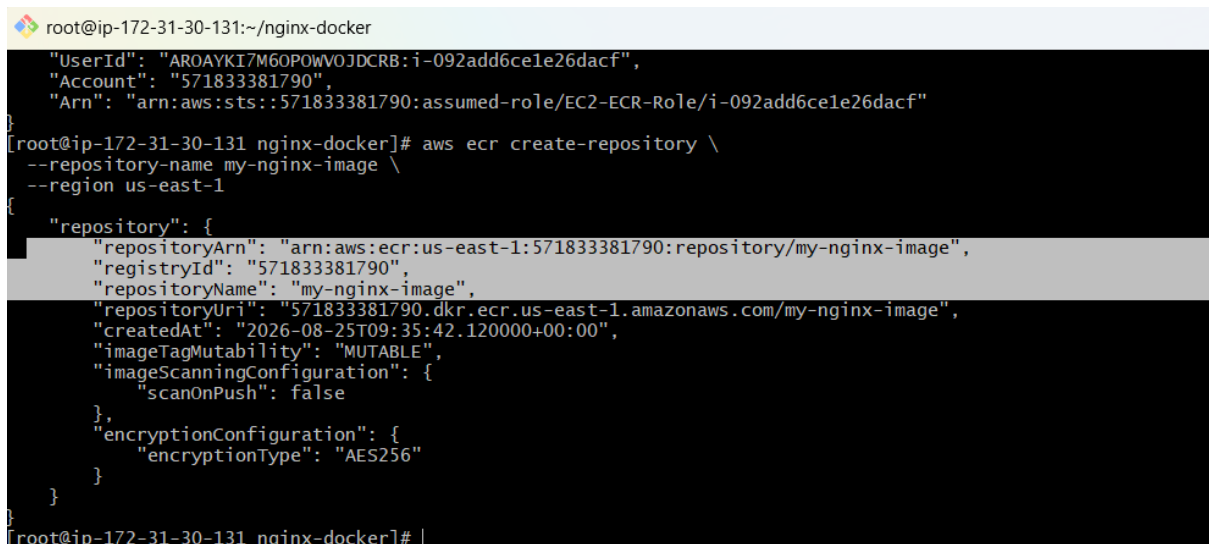
Now run:

```
aws ecr create-repository \
  --repository-name my-nginx-image \
  --region us-east-1
```

It should now succeed and return:

repositoryUri:

571833381790.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image



```
root@ip-172-31-30-131:~/nginx-docker
{"UserId": "AROAYKI7M6OP0WVOJDCRB:i-092add6ce1e26dacf",
 "Account": "571833381790",
 "Arn": "arn:aws:sts::571833381790:assumed-role/EC2-ECR-Role/i-092add6ce1e26dacf"}
[root@ip-172-31-30-131 nginx-docker]# aws ecr create-repository \
  --repository-name my-nginx-image \
  --region us-east-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:571833381790:repository/my-nginx-image",
    "registryId": "571833381790",
    "repositoryName": "my-nginx-image",
    "repositoryUri": "571833381790.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image",
    "createdAt": "2026-08-25T09:35:42.120000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
[root@ip-172-31-30-131 nginx-docker]# |
```

Login to ECR

```
aws ecr get-login-password --region us-east-1 | \
```

```
docker login --username AWS --password-stdin \
```

571833381790.dkr.ecr.us-east-1.amazonaws.com

Expected:

Login Succeeded

```
root@ip-172-31-30-131:~/nginx-docker
[root@ip-172-31-30-131 nginx-docker]# aws ecr get-login-password --region us-east-1 | \
docker login --username AWS --password-stdin \
571833381790.dkr.ecr.us-east-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-30-131 nginx-docker]# |
```

Tag your existing image

docker tag rajkumari2010/my-nginx-image:latest \

571833381790.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image:latest

Push

docker push \

571833381790.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image:latest

```
root@ip-172-31-30-131:~/nginx-docker
[root@ip-172-31-30-131 nginx-docker]# aws ecr get-login-password --region us-east-1 | \
docker login --username AWS --password-stdin \
571833381790.dkr.ecr.us-east-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-30-131 nginx-docker]# docker tag rajkumari2010/my-nginx-image:latest \
571833381790.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image:latest
[root@ip-172-31-30-131 nginx-docker]# docker push \
571833381790.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image:latest
The push refers to repository [571833381790.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image]
d0748c812789: Pushed
08d3bce09da0: Pushed
bba25e797da6: Pushed
latest: digest: sha256:559c1242b4b237d6f2ee2de7313e93b7df08494da389cc893c052b22d84cc0c1 size: 949
[root@ip-172-31-30-131 nginx-docker]# |
```

For Verification Open Amazon ECR

In the AWS Console, search for:

Amazon Elastic Container Registry (ECR)

Make sure the AWS region is:

US East (N. Virginia) — us-east-1

2. Open your repository

Go to:

ECR → Private registry → Repositories

You should see:

my-nginx-image

Click **my-nginx-image**.

The screenshot shows the AWS Elastic Container Registry (ECR) console. The left sidebar contains the navigation menu with 'Amazon ECR' selected, followed by 'Private registry' and 'Repositories'. The main content area is titled 'Private repositories (1)' and features a search bar with the placeholder 'Search by repository substring'. Below the search bar is a table with the following columns: Repository name, URI, Created at, Tag immutability, and Encryption type. A single repository, 'my-nginx-image', is listed in the table. The repository's URI is '571833381790.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image', and it was created on August 25, 2026, at 15:05:42 (UTC+05.5). The tag immutability is set to 'Mutable' and the encryption type is 'AES-256'. At the top right of the main content area, there are buttons for 'View push commands', 'Delete', 'Actions', and a prominent orange 'Create repository' button. The top of the console shows the AWS logo, a search bar, and the user's profile information (Rajkumari 571833381790).

Repository name	URI	Created at	Tag immutability	Encryption type
my-nginx-image	571833381790.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image	August 25, 2026, 15:05:42 (UTC+05.5)	Mutable	AES-256